

02-docker-realtime.md

Docker Image and Container (Simple Explanation)

1 ELI5 (Explain Like I'm 5)

Imagine you have a **cake recipe** and a **cake**.

- The **recipe** tells you how to make the cake.
- The **cake** is what you actually eat.

In Docker:

- **Image** = The **recipe** 📖
- **Container** = The **cake** 🍰

So when you use the recipe (image), you can create many cakes (containers).

Simple Points

- You must **start Docker** before using it.
- A **container needs an image** to be created.
- You must **start a container** before going inside it.
- When you run an **image**, Docker **creates a container automatically**.

2 ELI10 (Explain Like I'm 10)

Think of a **mobile app installation file (.apk)** and an **installed app**.

- **Docker Image** → Like the **APK file** (template or blueprint).
- **Docker Container** → Like the **installed and running app**.

The image contains everything needed to run the application: code, libraries, dependencies, and environment.

When you run the image, Docker creates a **container**, which is a running instance of that image.

Important Points

1. Docker Service Must Be Running

- You cannot use Docker commands until the Docker service is started.

2. Image is Required

- A **base image** is required to create a container.

3. Container Must Start First

- You cannot enter a container unless it is running.

4. Running an Image Creates a Container

- When you execute an image using `docker run`, Docker automatically creates and starts a container.

Example:

```
docker run ubuntu
```

This command:

- Pulls the **Ubuntu image** (if not available)
- **Creates a container**
- **Starts the container**

✓ Quick Summary

Component	Meaning
Image	Blueprint / Template
Container	Running instance of the image
docker run	Creates and starts a container

Docker Client and Docker Host

1 ELI5 (Explain Like I'm 5)

Imagine you want to order food.

- **You (customer)** tell the waiter what food you want.
- The **waiter** takes your request to the kitchen.
- The **kitchen** cooks the food.

In Docker:

- **Docker Client** → The **waiter** taking your request
- **Docker Host** → The **kitchen** where the work happens

So when you type a Docker command, the **Docker Client sends the request to the Docker Host**, and the host creates containers and runs them.

2 ELI10 (Explain Like I'm 10)

Docker works using two main parts:

Docker Client

The **Docker Client** is the tool you use to send commands to Docker.

Example command:

```
docker run ubuntu
```

This command is given from the **Docker Client**.

Docker Host

The **Docker Host** is the machine where Docker runs and where containers actually execute.

It contains:

- Docker Engine
- Docker Images
- Docker Containers

Flow

User Command → Docker Client → Docker Host → Container Runs

So the **client sends instructions**, and the **host performs the work**.

3 Feynman Technique (Simple but Complete Explanation)

Docker follows a **client-server architecture**.

Docker Client

The Docker Client is the interface used by users to interact with Docker.

When a user runs commands like:

```
docker build
docker pull
docker run
```

the Docker Client sends these requests to the **Docker Daemon**.

Docker Host

The Docker Host is the system that runs the **Docker Daemon (dockerd)**.

The daemon is responsible for:

- Building images
- Managing containers
- Pulling images from registries
- Running containers

Communication

The Docker Client communicates with the Docker Daemon using a **REST API**.

Architecture flow:

User

|



Docker Client

|



Docker Daemon (Docker Host)

|



Images / Containers / Networks / Volumes

Example Flow

1. User runs command:

```
docker run nginx
```

2. Docker Client sends request to Docker Daemon.

3. Docker Daemon:

- Checks for the image
- Pulls it if needed
- Creates a container
- Starts the container.

Quick Summary

Component	Role
Docker Client	Sends commands to Docker
Docker Host	Runs Docker Engine and containers
Docker Daemon	Executes Docker commands
Container	Running application

Example Command Flow

Command:

```
docker run nginx
```

Step-by-step process:

1. User executes the command in the **Docker Client**.
2. Docker Client sends the request to the **Docker Daemon**.
3. Docker Daemon checks if the **nginx image exists locally**.
4. If not available, it **pulls the image from Docker Registry**.
5. Docker Daemon **creates and starts the container**.



Docker Container Cheat Sheet (Practical)

◆ 1. View Containers (First thing you do)

Running containers

```
docker ps
```

All containers (running + stopped)

```
docker ps -a
```

Last created container

```
docker ps -l
```

Last 2 containers

```
docker ps -n 2
```

```
# Only exited containers
docker ps -f "status=exited"
```

♦ 2. Start / Stop (Basic operations)

```
# Stop a container (graceful)
docker stop <container>
```

```
# Force stop (immediate kill)
docker kill <container>
```

```
# Start a container
docker start <container>
```

```
# Restart
docker restart <container>
```

⚠ Real understanding

- `stop` = safe shutdown (waits)
- `kill` = immediate (can corrupt processes)

👉 Use `kill` only when things are stuck.

♦ 3. Bulk Operations (Where people mess up)

```
# Stop all running containers
docker stop $(docker ps -q)
```

```
# Start all containers
docker start $(docker ps -aq)
```

♦ 4. Remove Containers (Cleanup)

```
# Remove one container
docker rm <container>
```

```
# Force remove (even if running)
docker rm -f <container>
```

```
# Remove all stopped containers (SAFE)
docker container prune
```

Don't do this blindly

```
docker rm -f $(docker ps -aq)
```

👉 This wipes EVERYTHING → dangerous in shared environments

◆ 5. Logs & Debugging (Most useful in real life)

View logs

```
docker logs <container>
```

Follow logs (live)

```
docker logs -f <container>
```

Last 100 lines

```
docker logs --tail 100 <container>
```

◆ 6. Inspect & Details

Full details (JSON)

```
docker inspect <container>
```

See container size

```
docker ps -s
```

◆ 7. Rename Container

```
docker rename old_name new_name
```

◆ 8. Real-world workflow (IMPORTANT)

Scenario: App not working

```
docker ps # is it running?
```

```
docker logs -f <container> # check errors
```

```
docker restart <container> # quick fix
```

Scenario: Clean system

```
docker ps -a # see junk
```

```
docker container prune # remove stopped
```

Scenario: Everything stuck

```
docker stop $(docker ps -q)
docker start $(docker ps -aq)
```

Key Concepts (Don't skip)

Container states

- running 
 - exited 
 - paused 
-

IDs vs Names

You can use either:

```
docker stop my-container
docker stop 8f3a2c1d
```

Common mistakes (you were close to these)

-  using `docker ps` instead of `docker ps -q` in scripts
 -  deleting running containers without `-f`
 -  mixing full output instead of IDs
 -  overusing `kill`
-

Final advice

Right now you're learning commands.

That's not enough.

You need to think:

“What problem am I solving?”

Because in real DevOps:

- you debug failures
 - not memorize commands
-

If you want next step, I can give you:

 **Top 10 real production issues + exact docker commands to fix them**

That will level you up much faster than just docs.